SPRING BOOT NOTES

DAY 02 — MY SIMPLE STUDY NOTES

Writing Our First Spring Boot Application

Topics: localhost • ports • Spring Initializr • dependencies • project structure • REST controller • Tomcat

Beginner-friendly notes for learning and GitHub sharing.

1. What Are We Building?

We will create a small web application. The browser sends a request to the Spring Boot application, and the application returns a response.

```
Browser  -- GET /hello -->  Spring Boot App
Browser  <-- Hello World --  Spring Boot App
```

Expected result

```
URL:      http://localhost:8080/hello
Response: Hello World
```

2. IP Address and Port

IP address: identifies a machine on a network.
Port: helps the operating system send traffic to the correct application on that machine.

```
Example: 192.168.1.10:8080
IP address = 192.168.1.10
Port       = 8080
```

Protocol / App	Typical port
HTTP	80
HTTPS	443
Spring Boot default	8080

A port number helps the computer deliver incoming data to the correct application.

3. Localhost

localhost means your own computer. It commonly refers to the loopback address 127.0.0.1.

```
http://localhost:8080
= this computer, port 8080
```

4. DNS and Reverse Proxy

- DNS translates a domain name into an IP address.
- HTTP normally uses port 80; HTTPS normally uses port 443.
- A reverse proxy can receive a public request and forward it to an application running on another port.

```
Browser -> DNS -> IP address
Browser -> Reverse Proxy -> Application
```

5. Development Setup

- Install Java JDK and check it with:

```
java -version
```

- The lecture uses IntelliJ IDEA for Java and Spring Boot development.
- IntelliJ supports Maven, code navigation, auto-completion and debugging.

6. Spring Initializr

Spring Initializr is a project generator. It creates the basic structure of a Spring Boot project.

- Choose project type, language, Java version and Spring Boot version.
- Set project details and packaging.
- Add dependencies and generate the project.

7. Dependencies

A dependency is an external library needed by a project. In Maven projects, dependencies are written in pom.xml.

- Examples: Spring Web, MySQL Connector, Spring Data JPA, Spring Security, Lombok and Hibernate.

8. Spring Boot Version Types

- Stable release: tested and suitable for normal development.
- SNAPSHOT: work-in-progress version that may change.
- RC: Release Candidate; close to final but not yet the final release.

9. JAR Packaging

JAR means Java Archive. A Spring Boot executable JAR packages the application so it can be run directly.

10. Project Structure

File / Folder	Use
src/main/java	Java source code
src/main/resources	Configuration and resources
pom.xml	Maven configuration and dependencies
application.properties	Application settings
Main class	Starts the Spring Boot app

```
@SpringBootApplication
public class FirstSpringBootApplication {
    public static void main(String[] args) {
        SpringApplication.run(
            FirstSpringBootApplication.class, args);
    }
}
```

11. Controller and @RestController

A controller receives web requests. @RestController marks a class that can handle web requests and return response data.

```
@RestController
public class HelloController {
    // Add endpoint method here
}
```

12. Our First Endpoint

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello World";
    }
}
```

- @RestController marks the class as a web controller.
- @GetMapping("/hello") maps a GET request for /hello to the method.
- sayHello() is the method that runs.

- return "Hello World" sends the response text.

```
Open: http://localhost:8080/hello
Output: Hello World
```

13. Embedded Tomcat

With the Spring Web dependency, Spring Boot configures an embedded Tomcat server by default.

- Spring Boot starts the application.
- Tomcat listens for HTTP requests.
- Spring MVC maps the request to the controller method.
- The method response is returned to the browser.

14. Change the Port

Edit src/main/resources/application.properties to use another port:

```
server.port=9090
```

After restarting the app, open:

```
http://localhost:9090/hello
```

15. Request–Response Flow

```
1. Browser requests /hello
2. Tomcat receives the request
3. Spring MVC checks URL mappings
4. /hello matches sayHello()
5. Method returns "Hello World"
6. Response goes back to browser
```

16. Why Learn the Internals?

Spring Boot helps us start quickly, but understanding how it works helps us fix real problems.

If you get 404 Not Found

- Check the URL and port.
- Check whether the controller is detected.
- Check package structure and mapping.
- Confirm the application is running.

If startup is slow

- Check bean creation, database connections and startup logic.
- Check external service calls and auto-configuration.

17. Quick Revision

- IP address identifies a machine; a port identifies an application endpoint.
- localhost means your own computer.
- HTTP = 80, HTTPS = 443, Spring Boot default = 8080.
- Spring Initializr generates the starting project structure.
- Dependencies are declared in pom.xml.
- JAR means Java Archive.
- @RestController handles web requests.
- @GetMapping("/hello") maps GET /hello to a Java method.
- Embedded Tomcat listens for requests; Spring MVC routes them.
- server.port changes the configured port.

18. Practice Checklist

- ☐ Create a project using Spring Initializr.
- ☐ Open it in IntelliJ IDEA and run it.
- ☐ Create HelloController with GET /hello.
- ☐ Open http://localhost:8080/hello.
- ☐ Change the port to 9090 and test again.
- ☐ Explain the request–response flow in your own words.

Next Topics

Maven • Servlets • Tomcat • Spring Core • IoC • Dependency Injection • Beans • Spring MVC • Spring Boot auto-configuration

Personal study notes based on the provided Day 2 lecture PDF. Rewritten in simple English for learning and GitHub sharing.